



APRENDIZAGEM DE MÁQUINA

Otávio Calaça Xavier



Prazer, Otávio.



Otávio Calaça Xavier – otaviocx@ufg.br

Mestre e Doutorando em Ciência da Computação

Professor e Pesquisador no IFG e na UFG

Consultor em Arquitetura de Software e Machine Learning

Pai de gêmeos.

Sobre a Disciplina Aprendizagem de Máquina



- **Ementa:**

- Estudo dos fundamentos e aplicações de Aprendizagem de Máquina. Tipos de aprendizado: supervisionado, não supervisionado, semi-supervisionado e auto-supervisionado. Preparação e representação de dados, engenharia de atributos e construção de pipelines. Avaliação de modelos preditivos e descritivos. Modelos lineares, não lineares, redes neurais e técnicas de aprendizado profundo. Abordagens recentes de aprendizado com poucos rótulos e representações. Aplicações práticas em sistemas de recomendação e projetos integradores.

Sobre a Disciplina Aprendizagem de Máquina



- **Conteúdo Programático:**

- **Fundamentos de Aprendizado de Máquina e Representação de Dados**

- Tipos de aprendizado, features, similaridade, normalização e pipelines.

- **Aprendizado Supervisionado**

- Regressão, classificação, overfitting, regularização e métricas (MAE, MSE, F1, etc.).

- **Aprendizado Não Supervisionado**

- Clusterização, redução de dimensionalidade e métricas.

- **Aprendizado Semi e Auto-Supervisionado**

- Pseudo-labeling, label propagation, autoencoders e contrastive learning.



Sobre a Disciplina Aprendizagem de Máquina



- **Conteúdo Programático:**

- **Redes Neurais e Deep Learning**

- Neurônio artificial, funções de ativação, backpropagation e otimização.

- **Sistemas de Recomendação**

- Recomendação baseada em conteúdo, filtragem colaborativa e híbrida.

- **Projeto Final de *Machine Learning***

- Desenvolvimento e apresentação de um projeto completo de ML.



Introdução a Aprendizagem de Máquina com Python



Pandas



INF
INSTITUTO DE
INFORMÁTICA



PODER JUDICIÁRIO
Tribunal de Justiça do Estado de Goiás

Programa de
Residência em TI

O que é Inteligência Artificial?

“Simulação da inteligência humana em máquinas programadas para pensar como humanos e imitar suas ações.”

Oxford Languages

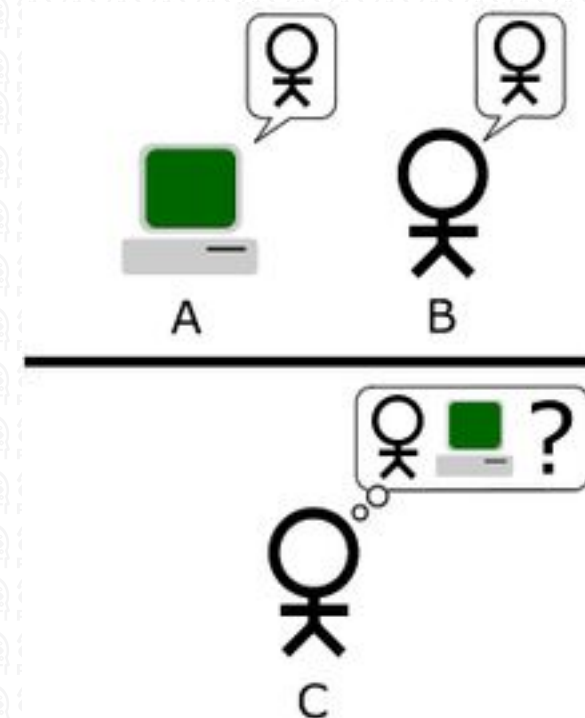
“Qualquer máquina que exiba características associadas à mente humana, como aprendizado e resolução de problemas.”

Wikipedia

O que é Inteligência Artificial?

O Teste de Turing

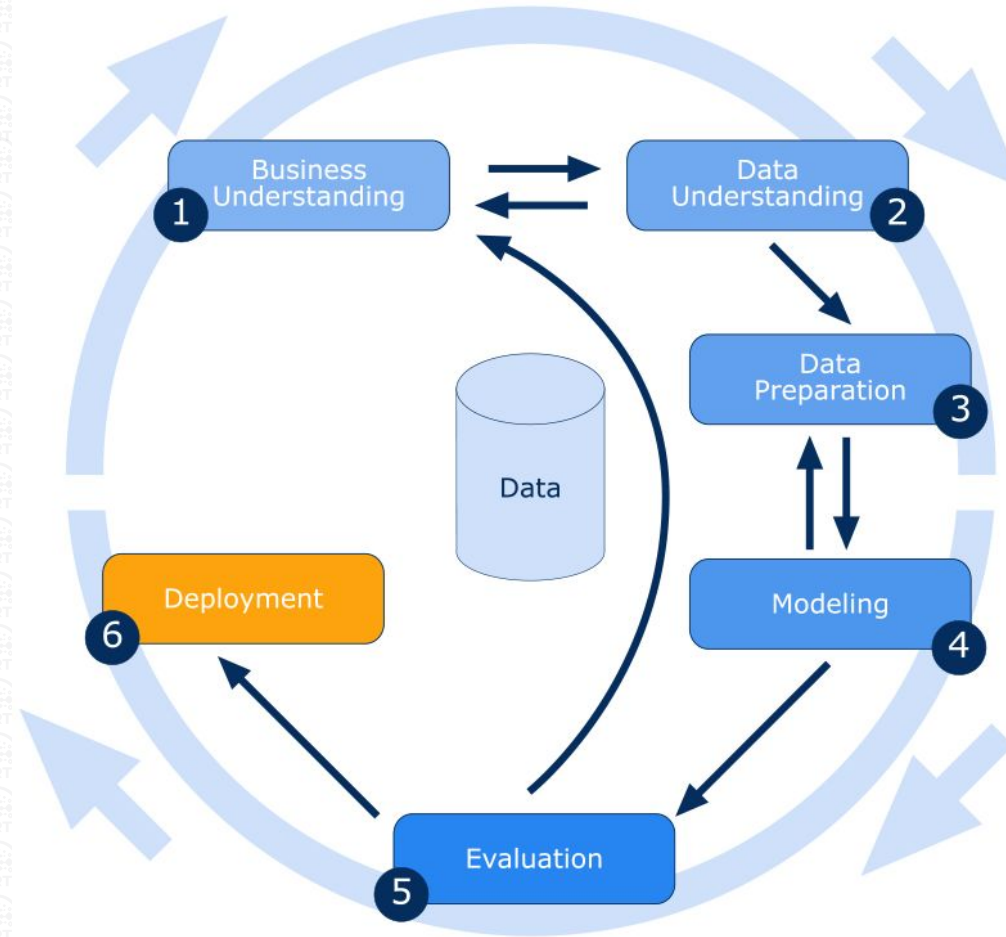
- Um humano conversa com outro humano e uma máquina e tem que dizer quem é a máquina.
- Introduzido por Alan Turing em seu artigo de 1950.
- Implementado desde 1990 até os dias atuais pelo Prêmio de Loebner.
- Recomendação: O Jogo da Imitação - 2014 - Filme



O que é Inteligência Artificial?

Data Mining

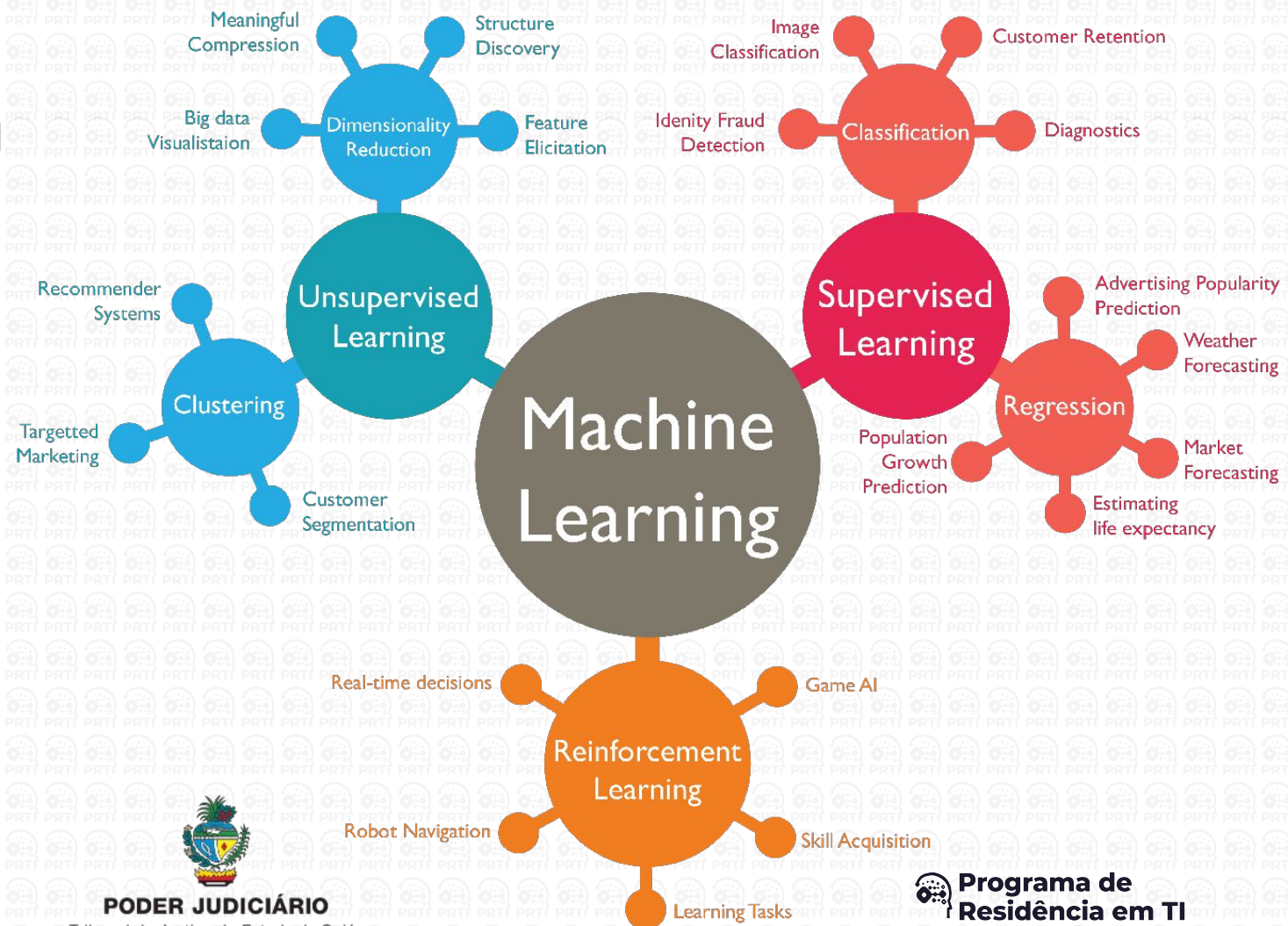
- Mineração de Dados
- Processo de análise da informação a partir de diferentes perspectivas.
- Inclui os processos de extração, entendimento, limpeza, normalização e análise dos dados.



O que é Inteligência Artificial?

Machine Learning

- Aprendizagem de Máquina
- Soluções que aprendem e extraem padrões em dados
- Conceito de 1960



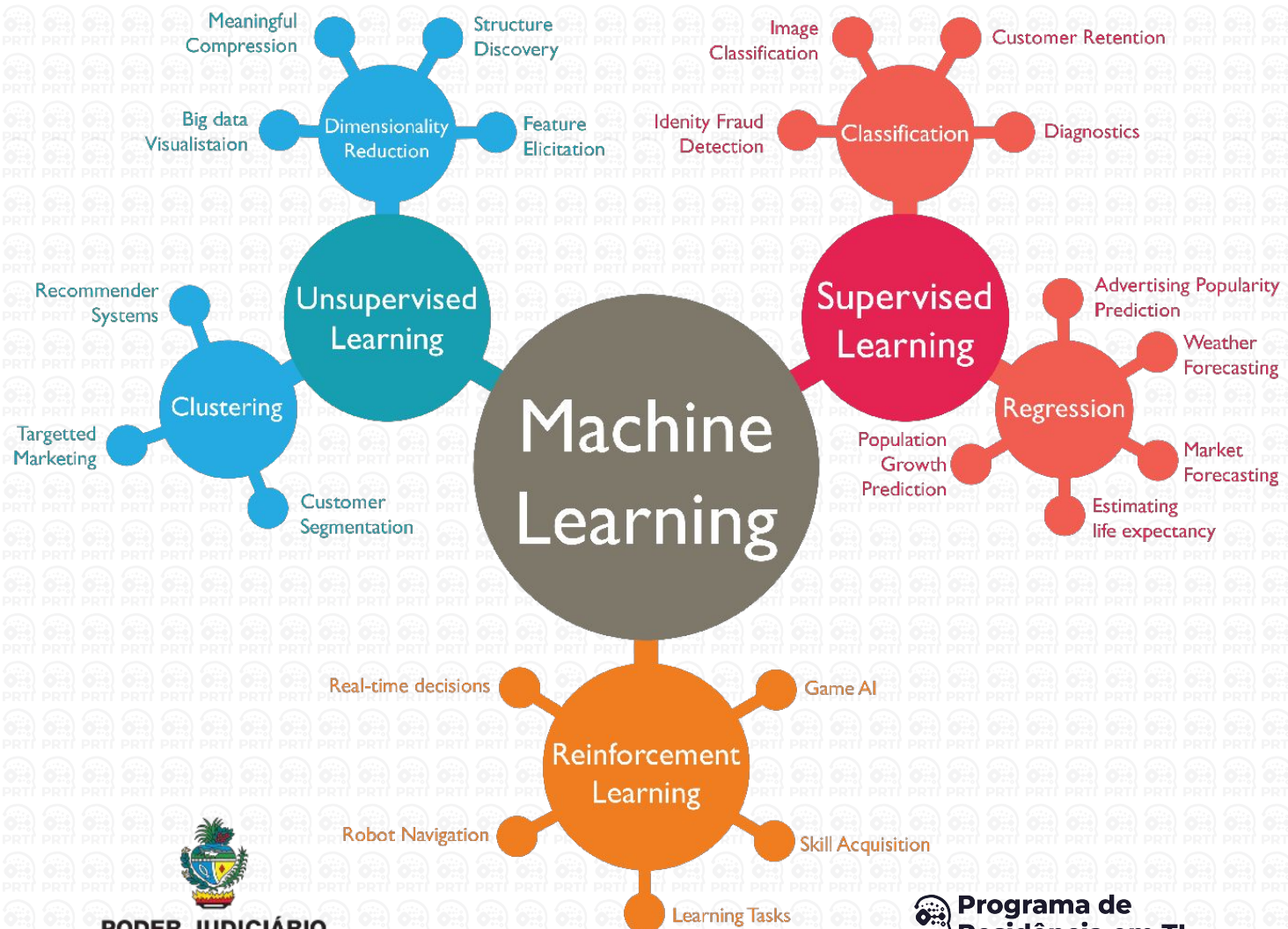
Introdução ao *Machine Learning*

Aprendizado

não-supervisionado

extração de padrões de dados desconhecidos. Visa inferir padrões através das características dos indivíduos

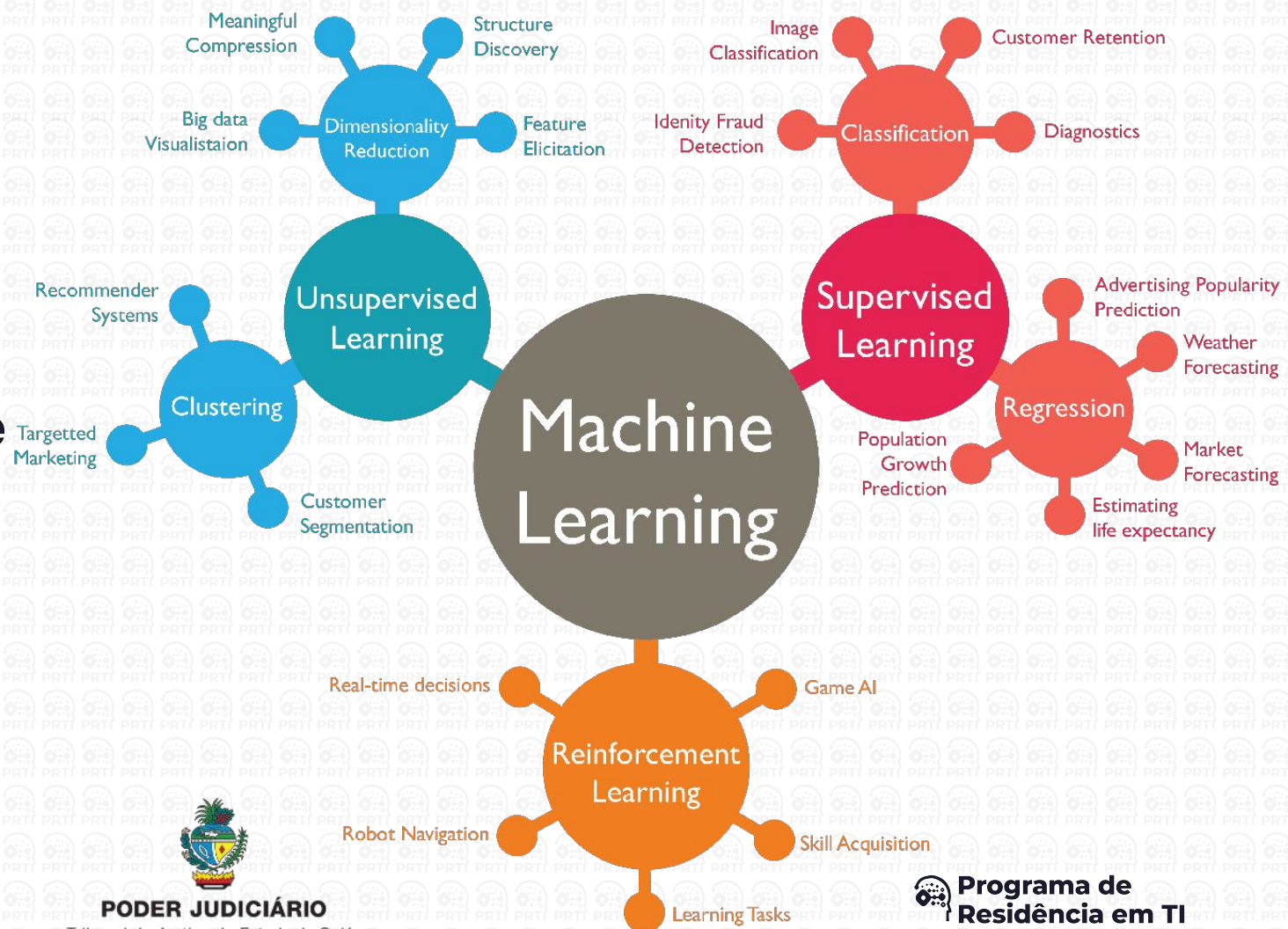
- Clusterização
- Redução de Dimensionalidade



Introdução ao *Machine Learning*

Aprendizado supervisionado aprende uma função que mapeia os dados de entrada na saída (por vezes chamada de ***label*** ou ***target***) através de exemplos de entrada e saída.

- Classificação
- Regressão

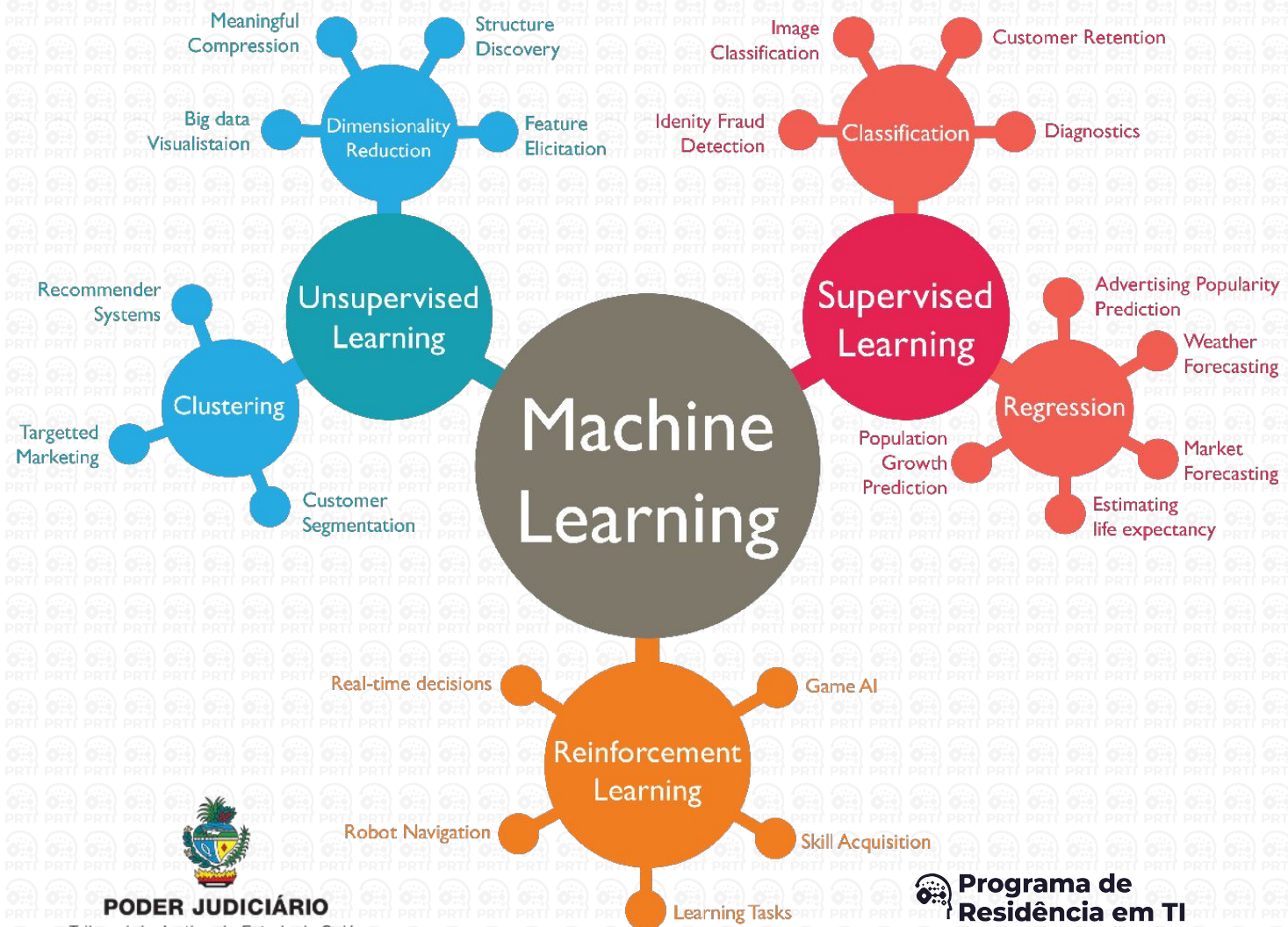


Introdução ao *Machine Learning*

Aprendizado por Reforço

Gera conhecimento sobre um problema a partir de “tentativas e erros” em um “mundo” bem definido.

- Jogos
- Composição de Músicas
- Pinturas



Introdução ao *Machine Learning*

Aprendizado Semi-Supervisionado

Combina dados rotulados e não rotulados no treinamento.

Usa o conhecimento dos dados rotulados para propagar rótulos ou inferir padrões nos demais.

Exemplos:

- Classificar imagens quando apenas 10% têm rótulo.
- Pseudo-labeling (usar previsões do modelo como novos rótulos).
- Algoritmos: Label Propagation, Label Spreading, Self-Training.

Vantagem: melhora o desempenho quando há poucos dados rotulados.

Introdução ao *Machine Learning*

Aprendizado Auto-Supervisionado

Gera seus próprios rótulos a partir dos dados.

Cria tarefas de pré-treinamento para aprender representações úteis (self-labels).

Exemplos:

- Predizer a próxima palavra (LLMs).
- Reconstruir uma imagem ou frase mascarada (autoencoders).
- Pré-treinar representações e transferir para outras tarefas.

Vantagem: dispensa rótulos humanos e melhora o aprendizado de representações.

Introdução ao *Machine Learning*

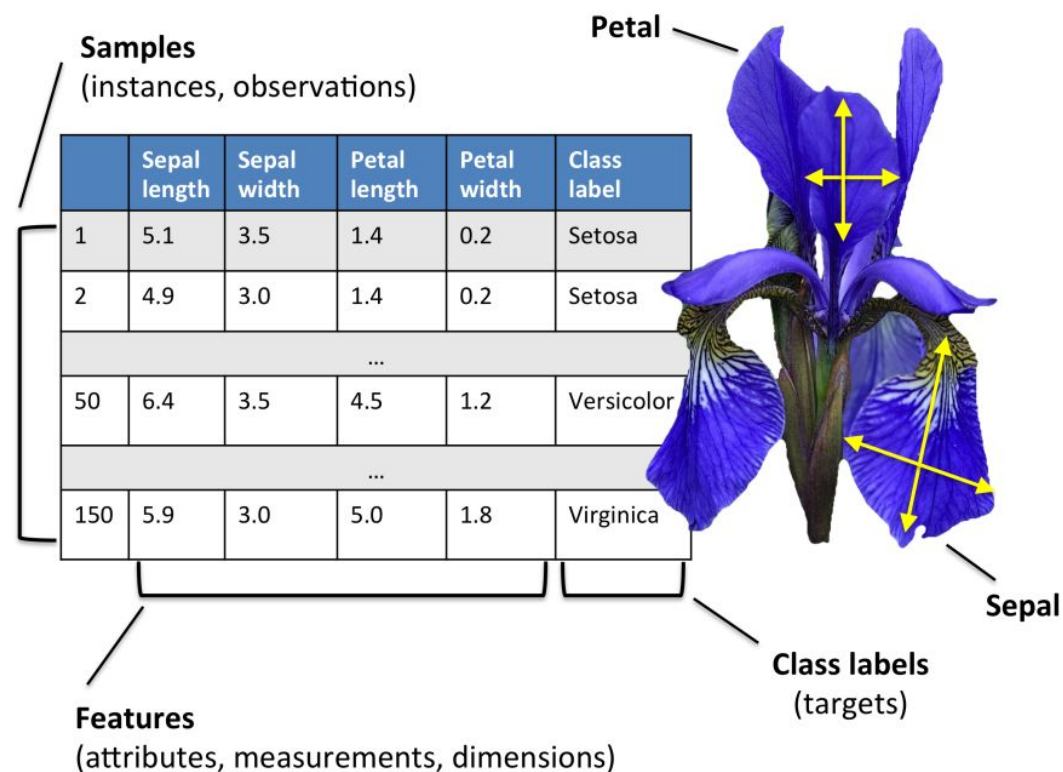
O que define qual categoria de algoritmos devemos utilizar é **o problema a ser resolvido e os dados** que temos acerca dele.

“Se a única ferramenta que você tem é um martelo, tudo começa a parecer com um prego.” - Abraham Maslow

ML - problema de negócio

Quando o problema pode ser resolvido com um algoritmo supervisionado (regressão ou classificação) é preciso ter em mente as **features** e o **target**.

Mas a vida não é o dataset do Kaggle.



ML - problema de negócio

Problema: Somos uma equipe de marketing e precisamos escolher grupos de clientes para direcionar nossas campanhas com base no perfil.

ML - problema de negócio

Problema: O setor de analytics detectou que no semestre passado houve um aumento de 20% no cancelamento de clientes. Quais estratégias podemos usar para mitigar esse indicador?

ML - problema de negócio

Problema: Precisamos calcular uma estimativa de vendas para os próximos dois meses para o produto A e o produto B, sendo que o produto B é novo na loja. O que indica?

ML - problema de negócio

Problema: Tenho um portal de notícias e desejo organizá-las de uma maneira personalizada para cada usuário. O que devo fazer?

ML - problema de negócio

Problema: Somos uma loja de roupas e queremos aumentar as vendas de nossos produtos pelo site. Nossos indicadores de acesso ao site são bons, mas o de conversão de compras não. Qual a sugestão?

ML - problema de negócio

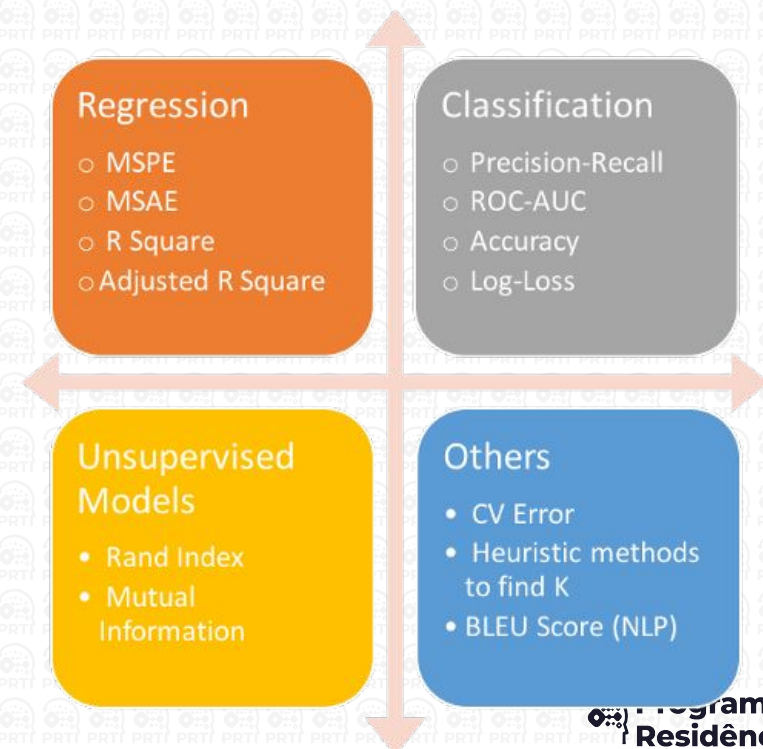
Problema: Somos uma empresa de cartão de crédito e precisamos criar um score de qualidade dos nossos clientes, esse score será utilizado para bonificações ou cancelamento do cartão.

*Quais são as **features** e qual é o **target**?*

ML - problema de negócio

Entender o problema faz parte da modelagem de uma solução de machine learning. Nem sempre as features e target estarão claras, só após entender o problema será possível escolher os algoritmos e métricas de avaliação que podem ser utilizados.

CLASSIFICATION	REGRESSION	CLUSTERING
Support Vector Machines	Linear Regression, GLM	K-Means, K-Medoids Fuzzy C-Means
Discriminant Analysis	SVR, GPR	Hierarchical
Naive Bayes	Ensemble Methods	Gaussian Mixture
Nearest Neighbor	Decision Trees	Neural Networks
	Neural Networks	Hidden Markov Model



Similaridade e Distância



Similaridade e Distância

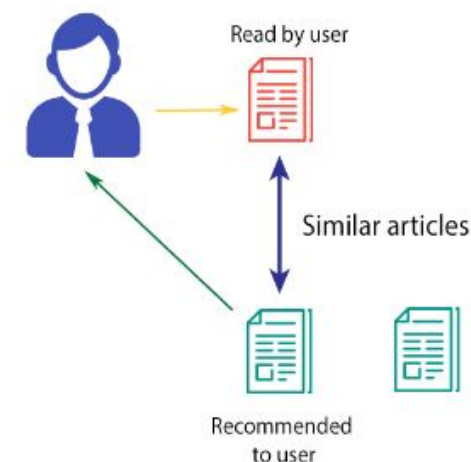
A similaridade é a base de muitos métodos de ML. Parte da ideia de que “se duas coisas (pessoas, empresas, produtos) são semelhantes em alguns aspectos, costumam compartilhar outras características também”.

Itens semelhantes podem gerar um cluster de perfil.

Classificar um novo conteúdo dado a semelhança de conteúdos conhecidos.

Recomendação de conteúdo semelhante em *RecSys*.

CONTENT-BASED FILTERING



Similaridade e Distância

Similaridade	Distância
Mede o quanto dois objetos são parecidos	Mede o quanto dois objetos são diferentes
Quanto mais parecido , maior o valor	Quanto mais diferente , maior o valor
Geralmente valor dentro do intervalo [0,1]	Geralmente valor no intervalo [0, X]

Normalmente as medidas satisfazem algumas **propriedades**:

- O objeto não é diferente de si próprio
 $d(x, x) = 0$ ou $s(x, x) = 1$
- Simetria
 $d(a, b) = d(b, a)$
- Positividade
 $d(a, b) \geq 0$

Similaridade e Distância



Dado o **espaço vetorial das *features***, é possível calcular a distância entre os indivíduos através de diferentes métricas, ex:

- **Euclidiana**

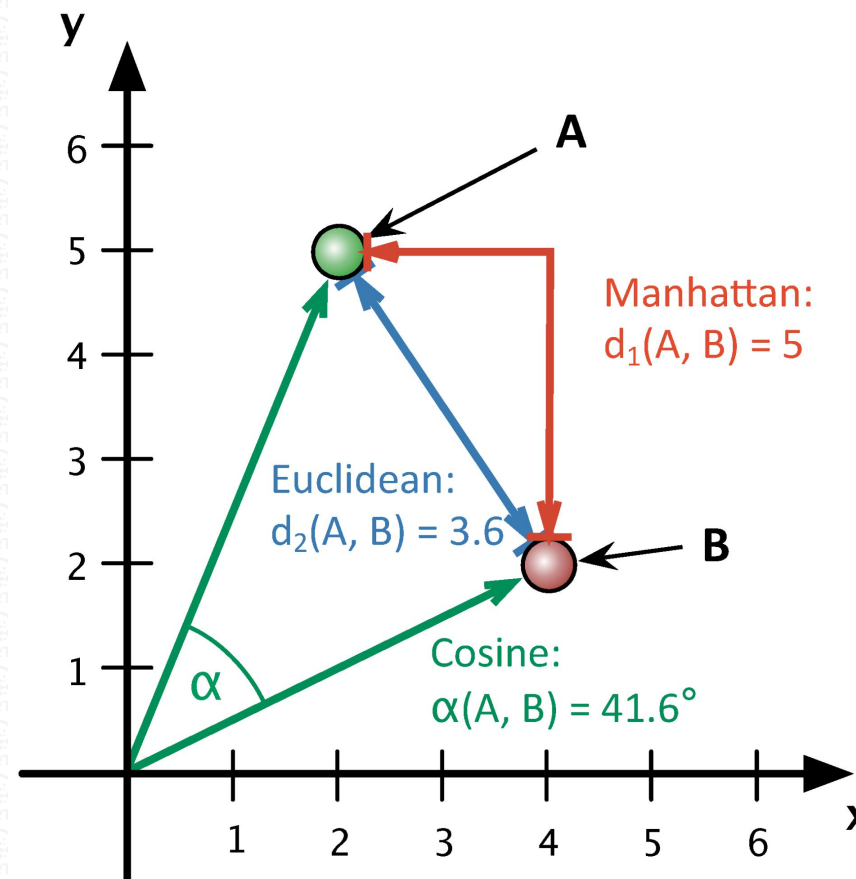
$$d(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

- **Manhattan**

$$d(A, B) = \sum_{i=1}^n |a_i - b_i|$$

- **Cosseno**

$$d(A, B) = \frac{A \cdot B}{\sqrt{A \cdot A} * \sqrt{B \cdot B}}$$



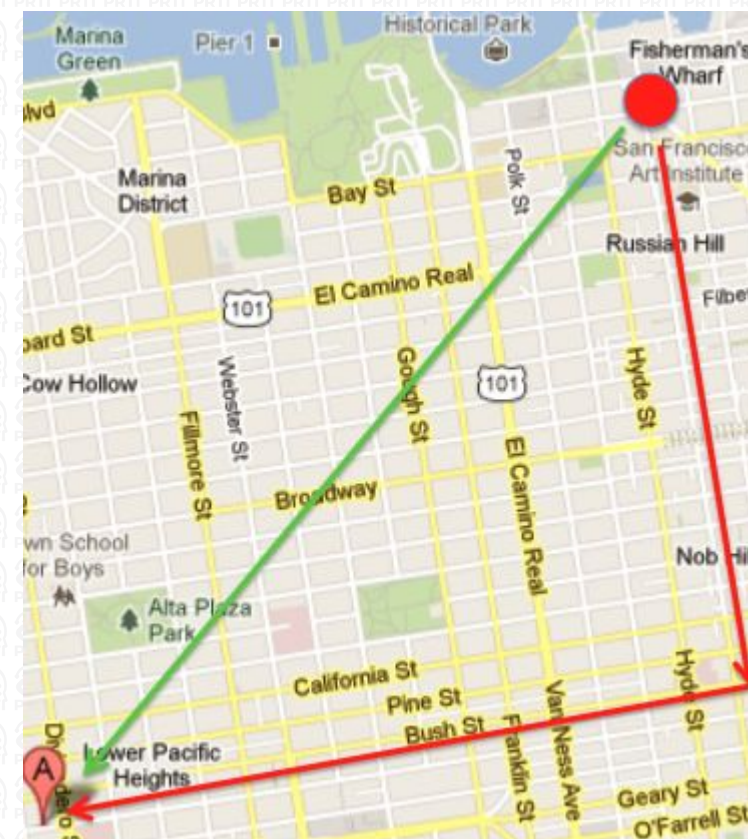
Similaridade e Distância



Qual a melhor métrica para **descrever a distância** entre dois endereços?

$A = [\text{lon}, \text{lat}]$

$B = [\text{lon}, \text{lat}]$



Similaridade e Distância

Similaridade não é só uma questão de distância e métrica, é uma questão de conceito.

- O que é um filme similar a outro?
Harry Potter é similar a **Senhor dos Anéis**?

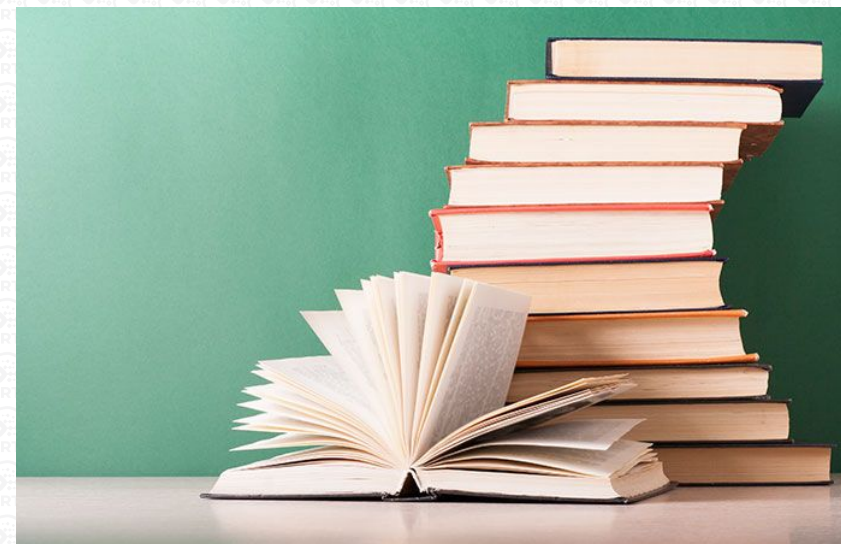


Similaridade e Distância



Similaridade não é só uma questão de distância e métrica, é uma questão de conceito.

- O que é um filme similar a outro?
Harry Potter é similar a Senhor dos Anéis?
- Com relação a livros, como definir similaridade entre livros e qual usar?



Similaridade e Distância



Similaridade não é só uma questão de distância e métrica, é uma questão de conceito.

- O que é um filme similar a outro?
Harry Potter é similar a Senhor dos Anéis?
- Com relação a livros, como definir similaridade entre livros e qual métrica usar?
- Música?



Similaridade e Distância

A definição de similaridade depende das *features* do indivíduo. Quando os dados já estão estruturados é mais fácil calcular a similaridade pois as features já foram definidas.

DADOS ESTRUTURADOS



bases de dados

DADOS NÃO ESTRUTURADOS



Variáveis (*features*)



Features:
1. Color: Radish/Red
2. Type : Fruit
3. Shape
etc...



Features:
1. Sky Blue
2. Logo
3. Shape
etc...



Features:
1. Yellow
2. Fruit
3. Shape
etc...

Variáveis (*features*)

É preciso definir as **features** para depois calcular similaridade, ou seja, um indivíduo é definido pelo seu vetor de features: David = [37,4000,2]

Cliente	Idade	Renda	Cartões	Resposta (alvo)	Distância euclidiana
David	37	4000	2	?	0
John	35	3000	3	Sim	15,16
Rachael	22	3000	2	Não	15
Ruth	63	8000	1	Não	152,23
Jefferson	59	10000	1	Não	122
Norah	25	2500	4	Sim	15,74

Variáveis (*features*)

Features não numéricas devem ser transformadas para realizar o cálculo de similaridade. É preciso transformar para que o *vetor de features* continue numérico:

David = [?, 37,4000,2]

Cliente	Emprego	Idade	Renda	Cartões	Resposta (alvo)	Distância euclidiana
David	Professor	37	4000	2	?	
John	Professor	35	3000	3	Sim	
Rachael	Programador	22	3000	2	Não	
Ruth	Gerente de TI	63	8000	1	Não	
Jefferson	Gerente de TI	59	10000	1	Não	
Norah	Professor	25	2500	4	Sim	

Variáveis (*features*)

Os dados podem ter naturezas diferentes e necessitam ser formatados de forma adequada para que os algoritmos “entendam”.

- **Categóricas (string)**
 - **Nominal:** Gênero, estado civil, tipos
 - **Ordinal:** Escolaridade, dia do Mês...
- **Discretas (int)**
 - Idade, qtd de filhos
- **Contínuas (float)**
 - Valores dentro de um intervalo

Emprego	Idade	Renda
Professor	37	4000
Professor	35	3000
Programador	22	3000
Gerente de TI	63	8000
Gerente de TI	59	10000
Professor	25	2500

Variáveis (*features*)

Label Encoder: Transforma categorias em inteiros, leva a variável categórica para o domínio discreto.

- Método Simples
- Adicionar informação de ordem

Emprego	EmpregoEnc
Professor	1
Professor	1
Programador	2
Gerente de TI	3
Gerente de TI	3
Professor	1

Variáveis (*features*)

OneHotEncoder: Transforma categorias em vetores binários, leva uma variável categórica para diferentes variáveis binárias.

- Resolve o problema da ordem
- Aumenta a features

Emprego	Professor	Programador	Gerente de TI
Professor	1	0	0
Professor	1	0	0
Programador	0	1	0
Gerente de TI	0	0	1
Gerente de TI	0	0	1
Professor	1	0	0

Variáveis (*features*)

A codificação das features faz parte da etapa de ***data preparation***. Tem o objetivo de extrair mais padrões nos dados.

Emprego	EmpregoEnc
Professor	1
Professor	1
Programador	2
Gerente de TI	3
Gerente de TI	3
Professor	1

Emprego	=Professor	=Programador	=Gerente de TI
Professor	1	0	0
Professor	1	0	0
Programador	0	1	0
Gerente de TI	0	0	1
Gerente de TI	0	0	1
Professor	1	0	0

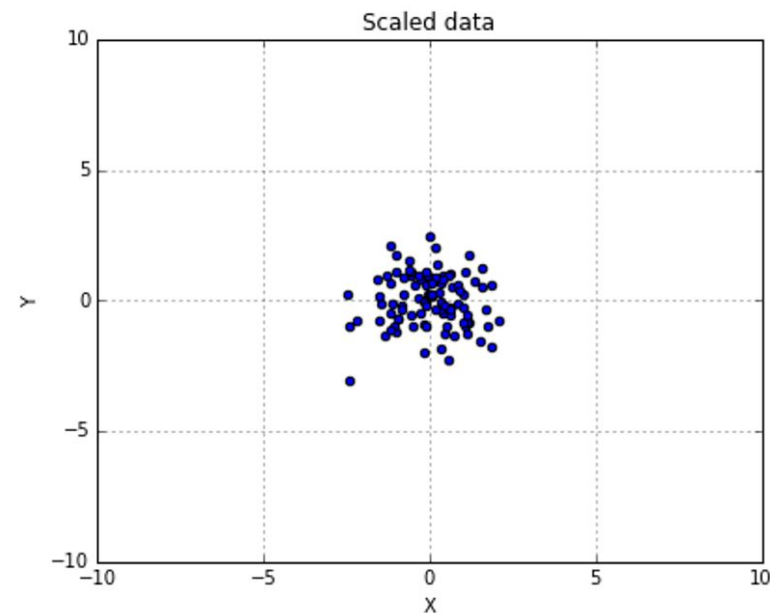
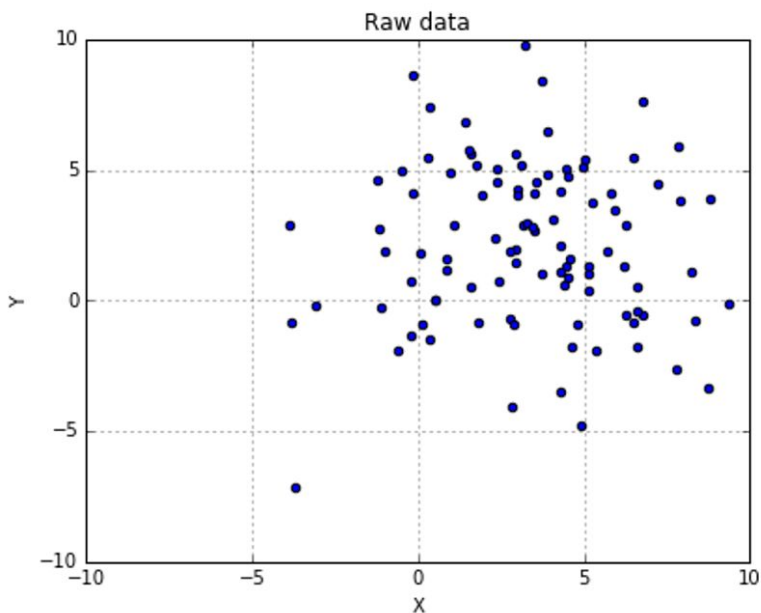
Variáveis (*features*)

Após o processo de transformação das variáveis o vetor fica completamente numérico.

David = [1, 0, 0, 37, 4000, 2]

Cliente	=Professor	=Programador	=Gerente de TI	Idade	Renda	Cartões
David	1	0	0	37	4000	2
John	1	0	0	35	3000	3
Rachael	0	1	0	22	3000	2
Ruth	0	0	1	63	8000	1
Jefferson	0	0	1	59	10000	1
Norah	1	0	0	25	2500	4

Normalização



Normalização

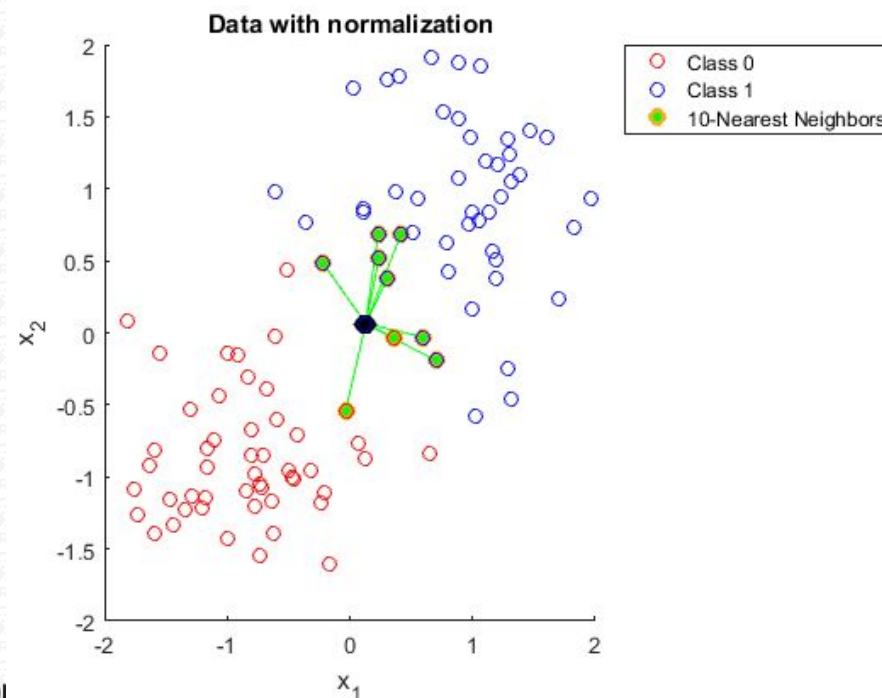
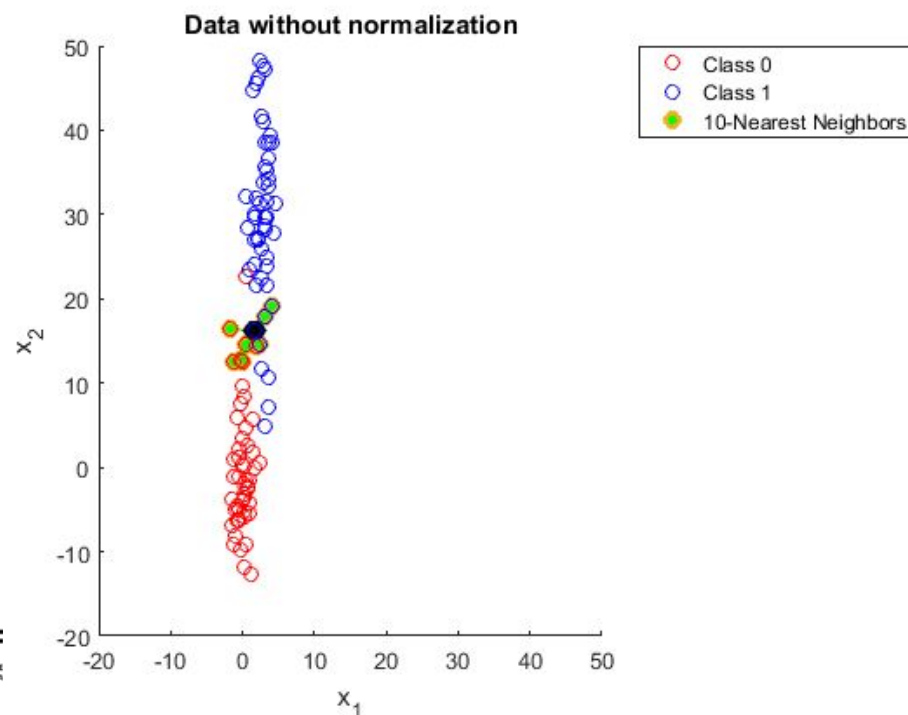
Diferentes algoritmos de ML podem ser influenciados negativamente por dados não normalizados. De forma geral, os dados devem estar na mesma faixa de valores, isso evita que uma dimensão se sobreponha em relação às outras.

Cliente	=Professor	=Programador	=Gerente de TI	Idade	Renda	Cartões
David	1	0	0	37	4000	2
John	1	0	0	35	3000	3
...	..	..	..	..	..	..

Qual dimensão está sobrepondo as outras?

Normalização

Com os dados normalizados a fronteira de decisão do algoritmo fica melhor distribuída, o que reflete na performance do mesmo.



Normalização

Normalização por Máx-Min utiliza os valores máximos e mínimos para normalizar **linearmente** os dados entre [0, 1].

- Muito sensível aos outliers.
- Em alguns casos o máximo e o mínimo não são conhecidos.

$$f(x) = \frac{x - \min(x)}{\max(x) - \min(x)}$$

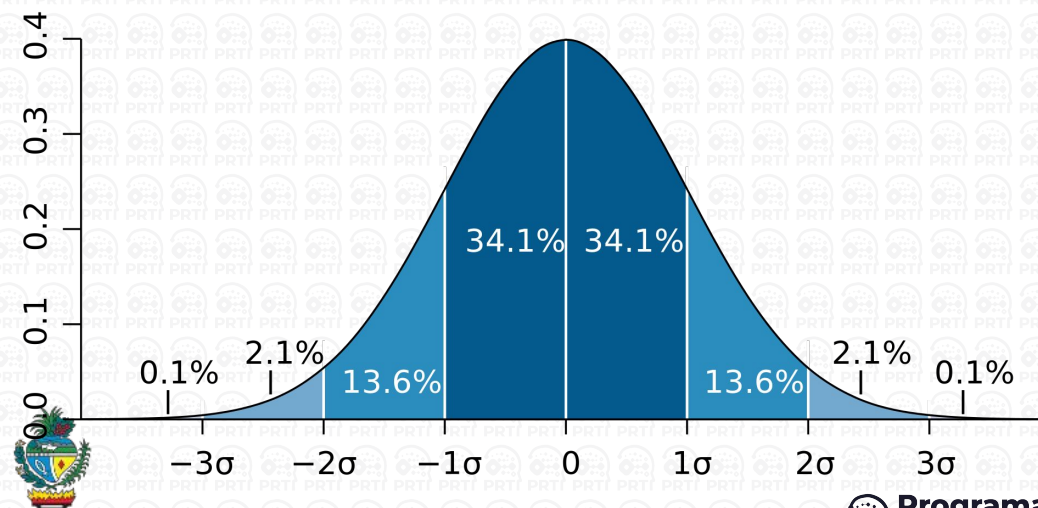
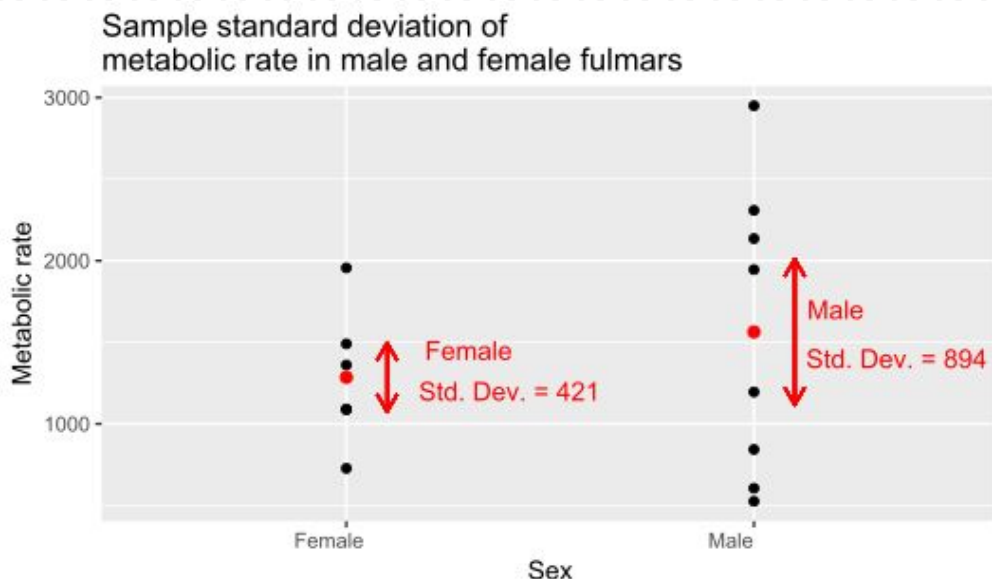
Exemplo de features boas e ruins para usar com Normalização de Máx-Min?

Normalização

Normalização por Desvio Padrão (z-score) os dados são normalizados no entorno da média e do desvio padrão, valores ficam em torno do zero.

- Pouco sensível a outliers.
- Funciona bem na maioria dos casos.

$$f(x) = \frac{x - \bar{x}}{\sigma_x}$$



Normalização

O processo de normalização é uma etapa do pipeline de ML assim como Engenharia de Features.

Cliente	=Professor	=Programador	=Gerente de TI	Idade	Renda	Cartões
David	1	0	0	37	4000	2
John	1	0	0	35	3000	3
...	..	..	..	..	..	..

Transformação
 $df = \text{normalize}(df)$

Cliente	=Professor	=Programador	=Gerente de TI	IdadeNorm	RendaNorm	CartõesNorm
David	1	0	0	1	2,3	0,4
John	1	0	0	0,8	1,8	0,6
...	..	..	..	..	..	..

Normalização



```
class sklearn.preprocessing. MinMaxScaler (feature_range=(0, 1), copy=True)
```

[\[source\]](#)

```
class sklearn.preprocessing. StandardScaler (copy=True, with_mean=True, with_std=True)
```

[\[source\]](#)

Examples

```
>>> from sklearn.preprocessing import MinMaxScaler

>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler(copy=True, feature_range=(0, 1))
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.  0. ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.  1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0.  ]
```

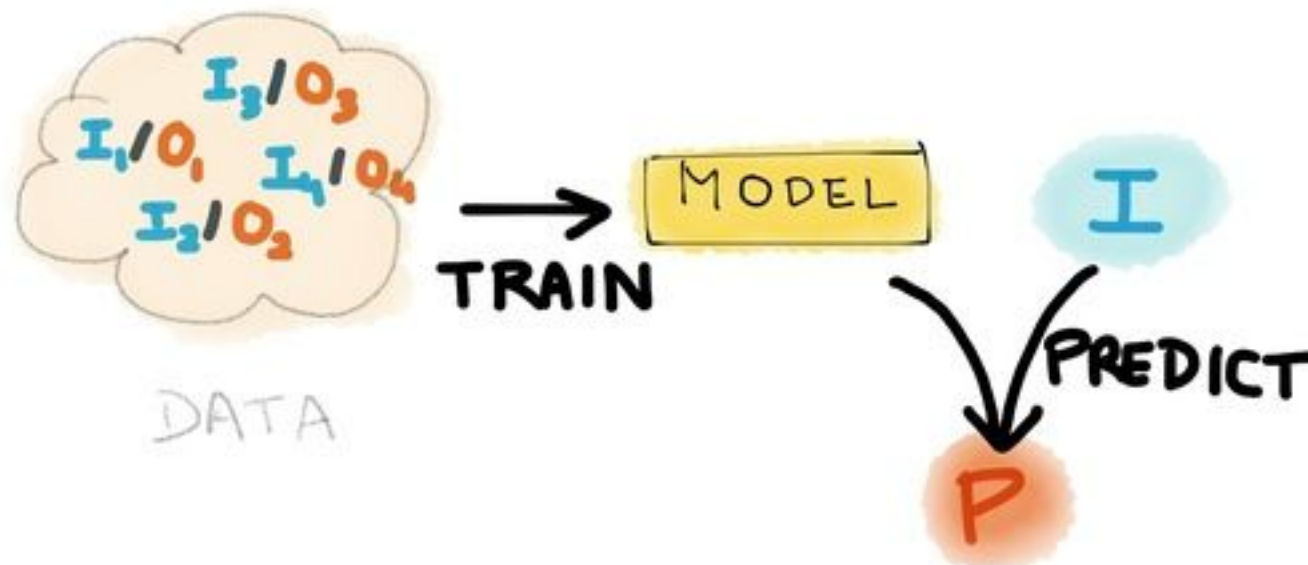
Examples

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> print(scaler.fit(data))
StandardScaler(copy=True, with_mean=True, with_std=True)
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Pipeline de Machine Learning

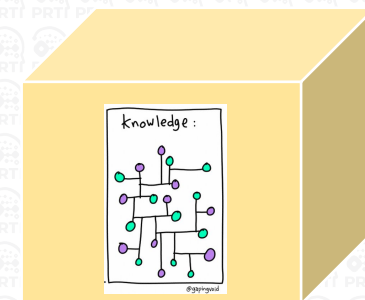


Pipeline de Machine Learning

Quando o modelo for utilizado em produção, esse é o dataset/input que irá receber?

=Professor	=Programador	=Gerente de TI	IdadeNorm	RendaNorm	CartõesNorm
1	0	0	1	1,2	1
1	0	0	0,2	2,3	0.3
..	..	..	..	..	..

Resposta (alvo)
Sim
Não
...



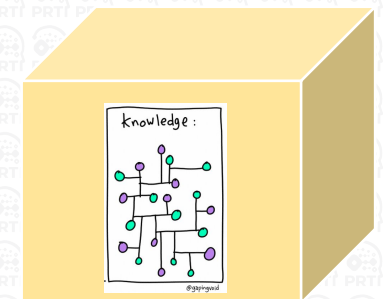
Modelo $f(X) = y$

Pipeline de Machine Learning

não.... o dado deve ser o mais próximo do original possível.

Cliente	Emprego	Idade	Renda	Cartões
David	Professor	37	4000	2
John	Professor	35	3000	3
...	...	...	...	...

Resposta (alvo)
Sim
Não
...



Modelo $f(X) = y$

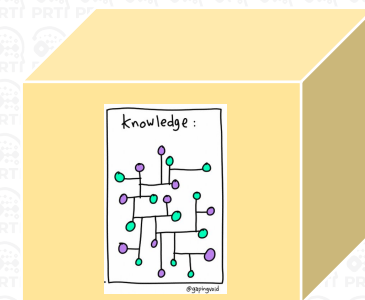
Pipeline de Machine Learning

A necessidade do pipeline em dados **não estruturados** fica mais evidente com texto. A entrada do modelo/pipeline sempre é o texto “cru”.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed auctor scelerisque quam, vitae aliquet arcu. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Proin rutrum porttitor turpis vel dignissim. Quisque facilisis quis ligula non eleifend. Nulla facilisi. Donec vita...

Resposta
(alvo)

Não



Modelo $f(X) = y$

Pipeline de Machine Learning

O “*modelo de machine learning*” não é o algoritmo (KNN, Regressão, MLP...), mas o pipeline completo de Transformações + Algoritmo.

Pipeline

Treinamento



Inferência



Pipeline de Machine Learning



Código com o pipeline de treino e inferência são muito semelhantes, compartilha principalmente as transformações nos dados, ou seja, toda a etapa de **Data Preparation**

Pipeline Treino

```
# Data Treino
X_data, y_data = [...]

# Transformacao 1
X_data = extract_date_info(X_data)

# Transformacao 2
norm_val, X_data = normalize_data(X_data,
                                  train=True)

# Train
model = KNeighborsClassifier()
model.fit(X_data)

# Avaliação
model.score(X_data, y_data)

# Save
joblib.dump(model, 'model.pkl')
```

Pipeline

```
# Dado Novo
X_data = [...]

# Transformacao 1
X_data = extract_date_info(X_data)

# Transformacao 2
X_data = normalize_data(X_data, norm_val=norm_val,
                        train=False)

# Train
model = load_model('model.pkl')

#Inferencia
model.predict(X_data)
```

Pipeline de Machine Learning



O Sklean tem uma estrutura para trabalhar direto com o pipeline de **Data Preparation + Model**

Pipeline Treino

```
# Data Treino
X_data, y_data = [...]

# Pipeline
pip1 = Pipeline([
    ('transf', ExtractDateInfo()),
    ('scaler', MinMaxScaler()),
    ('clf', KNeighborsClassifier())
])

# Train
pip1.fit(X_data)

# Avaliação
pip1.score(X_data, y_data)

# Save
joblib.dump(model, 'model.pkl')
```

Pipeline Inferência

```
In [ ]: # Data Treino
X_data, y_data = [...]

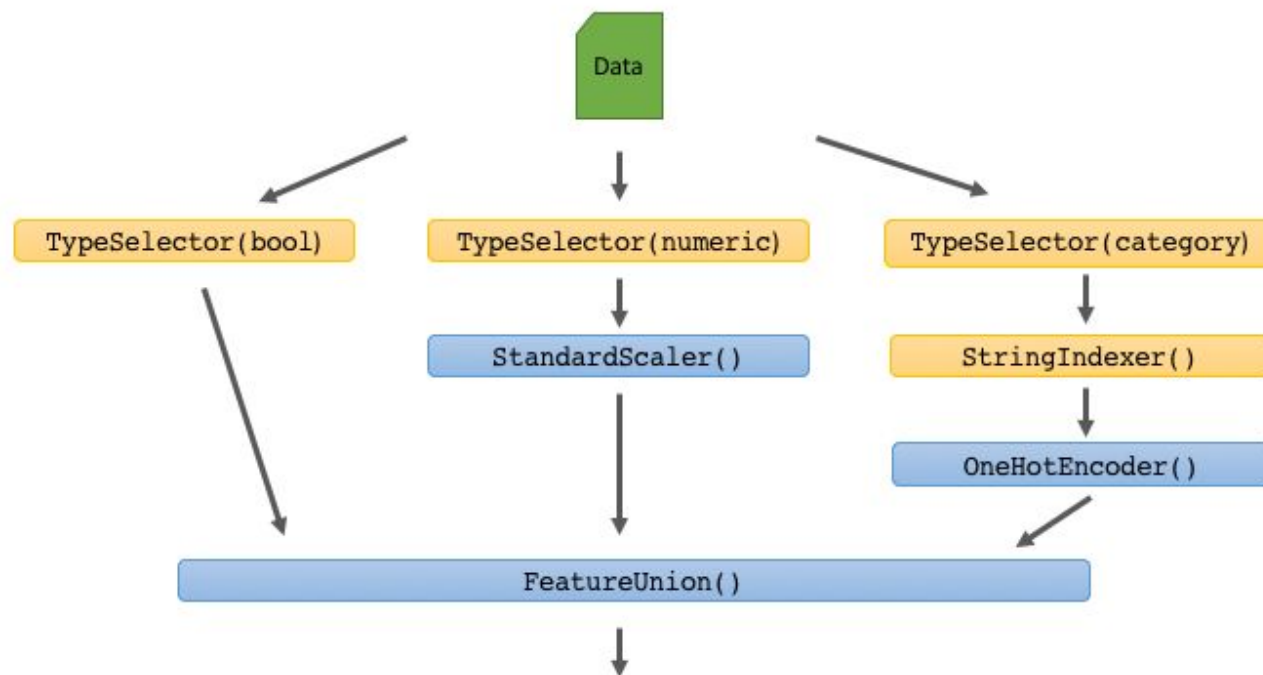
# Load Pipeline
pip1 = load_model('model.pkl')

# Inferencia
pip1.predict(X_data)
```

Pipeline de Machine Learning

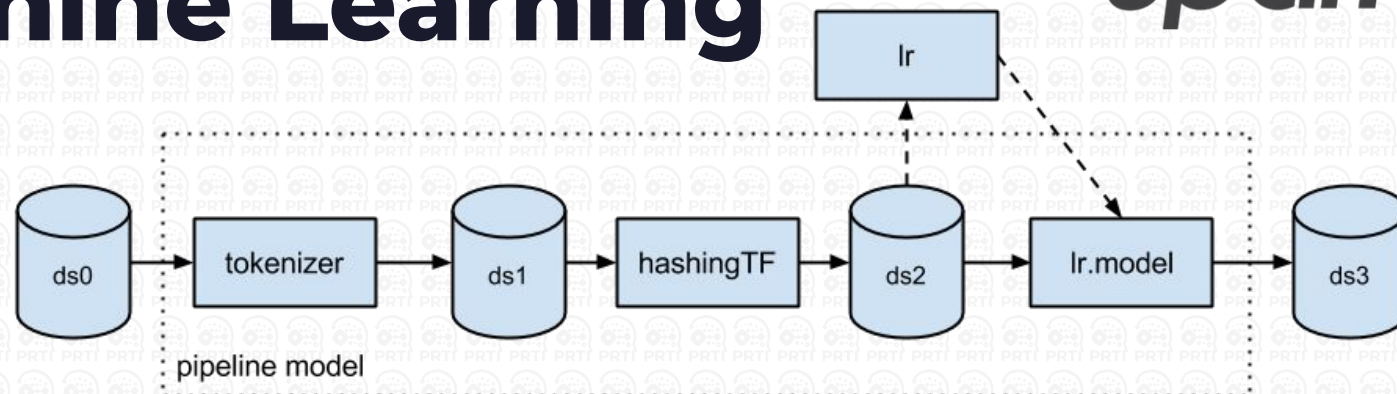


A estrutura de **Pipeline** é extremamente versátil e pode ser combinada para criar transformações específicas por features ou grupo de features.



Pipeline de Machine Learning

O **Spark** também conta com uma estrutura de Pipeline semelhante ao do Sklearn.



```
from pyspark.ml.feature import Tokenizer
from pyspark.ml.feature import CountVectorizer
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

tokenizer = Tokenizer(inputCol="texto", outputCol="words") # Tokenizer words
wordCount = CountVectorizer(inputCol="words", outputCol="features") # word count
lr = LogisticRegression(maxIter=10) # Classificador

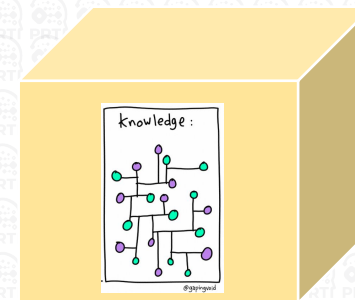
# Pipeline ML
pipeline = Pipeline(stages=[tokenizer, wordCount, lr])

# Fit the pipeline to training documents.
model = pipeline.fit(df)

# Predict in new dataset
df_pred = model.transform(df)
```

Pipeline de Machine Learning

No fim, a nossa caixinha que chamados de “**Modelo**” é a **materialização desse pipeline de dados** que pode ser usado em um ambiente diferente.



Pipeline $f(X) = y$

Pipeline de Machine Learning - CRISP-DM

Assim como em Engenharia de Software, existem vários modelos de processo para construção de um **pipeline de um projeto ML**.

Um dos mais utilizados é o **CRISP-DM**: *Cross-industry standard process for data mining*.

Semelhante aos métodos ágeis, é iterativo e incremental, possui etapas bem definidas e entregas rápidas e periódicas.



Pipeline de Machine Learning - CRISP-DM

O processo de construção de um pipeline de ML **está muito ligado ao problema de negócio** que fundamenta as escolhas de preparação de dados, algoritmos e avaliação da solução.



Show me the code

—

... in Python, please!

https://colab.research.google.com/drive/1EYE-YA63U8wGIMS_fFTqDd_UecUhpXx?usp=sharing

INF

INSTITUTO DE
INFORMÁTICA



UFMG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Atividades



1. Analisar o Notebook do Titanic e modificá-lo em busca de melhorar o resultado do modelo de *machine learning*.

